

Fundamentos y Evolución de HTTP y HTTPS

Alumnos: Abelle-Trovato-Romano

1. Conceptos Fundamentales del Protocolo HTTP

- 1.1 Definición y Propósito Fundamental del Protocolo
- 1.2 Arquitectura Cliente-Servidor y Esquema de Petición-Respuesta
- 1.3 Naturaleza del Protocolo: HTTP como Protocolo sin Estado
- 1.4 Actores del Sistema: Agentes de Usuario y Servidores de Origen

2. Historia y Evolución de las Versiones

- 2.1 HTTP/0.9 (1991): La Versión Original
- 2.2 HTTP/1.0 (1996): Introducción de Cabeceras y Metadatos
- 2.3 HTTP/1.1 (1999): Conexiones Persistentes y el Estándar RFC 2616
- 2.4 HTTP/2 (2015): Multiplexación y Optimización
- 2.5 HTTP/3 (2018): El Cambio hacia QUIC y UDP

3. Funcionamiento Interno: Mensajes y Recursos

- 3.1 Representaciones de Recursos
- 3.2 Estructura y Sintaxis de los Mensajes
- 3.3 Gestión de Sesiones mediante Cookies
- 3.4 Rol de los Intermediarios: Proxies, Puertas de Enlace y Túneles

4. Semántica de los Métodos de Petición

- 4.1 Clasificación: Seguridad e Idempotencia
- 4.2 Métodos de Recuperación: GET y HEAD
- 4.3 Métodos de Envío y Procesamiento de Datos: POST y PUT

4.4 Otros Métodos: DELETE, CONNECT, OPTIONS y TRACE

5. Códigos de Estado y Respuestas del Servidor

5.1 Clasificación por Categorías

5.2 Respuestas Informativas (1xx) y de Éxito (2xx)

5.3 Redirecciones (3xx)

5.4 Errores del Cliente (4xx) y del Servidor (5xx)

6. Funcionalidades Avanzadas del Protocolo

6.1 Negociación de Contenido

6.2 Peticiones Condicionales y Validadores

6.3 Peticiones de Rango y Transferencia Parcial

6.4 Almacenamiento en Caché

7. El Protocolo Seguro: HTTPS

7.1 Diferencias entre HTTP y HTTPS

7.2 Funcionamiento de TLS: Establecimiento de Conexión y Cifrado

7.3 Infraestructura de Clave Pública (PKI) y Certificados X.509

8. Consideraciones de Seguridad y Privacidad

8.1 Ataques de Intermediario (Man-in-the-Middle)

8.2 Vulnerabilidades Comunes

8.3 Privacidad y Huella Digital del Navegador

9. Documentación Técnica y Estandarización

9.1 El Rol de los Documentos RFC

9.2 RFC 9110: La Especificación Actual de la Semántica HTTP

9.3 Limitaciones de HTTP/1.1 y el Origen de HTTP/2

Resumen

El presente documento ofrece un resumen estructurado de los fundamentos y la evolución del Protocolo de Transferencia de Hipertexto (HTTP) y su variante segura HTTPS. Se abordan los aspectos definitorios del protocolo, su arquitectura cliente-servidor, las versiones históricas desde HTTP/0.9 (1991) hasta HTTP/3 (2018), el funcionamiento interno de los mensajes, la semántica de los métodos de petición, los códigos de estado, las funcionalidades avanzadas como la negociación de contenido y el almacenamiento en caché, y los mecanismos de seguridad proporcionados por la capa TLS. También se examinan las consideraciones de seguridad y privacidad, así como la estandarización del protocolo a través de los documentos RFC de la IETF.

Fundamentos y Evolución de HTTP y HTTPS

1. Conceptos Fundamentales del Protocolo HTTP

1.1 Definición y Propósito Fundamental del Protocolo

El Protocolo de Transferencia de Hipertexto (HTTP, por sus siglas en inglés Hypertext Transfer Protocol) es el protocolo de comunicación fundamental de la capa de aplicación en el modelo TCP/IP que permite la transferencia de información en la World Wide Web. HTTP se define como un protocolo orientado a transacciones, diseñado para sistemas de información de hipermedia distribuidos y colaborativos.

Su función principal es establecer las reglas de sintaxis y semántica que utilizan los diversos elementos de software —clientes, servidores y proxies— para comunicarse de manera efectiva en la red. Entre sus propósitos clave se encuentran la abstracción de la implementación, que oculta los detalles de cómo se implementa un servicio presentando una interfaz estándar independiente del tipo de recurso; y la transferencia de representaciones, mediante la cual, en lugar de transferir el recurso en sí, HTTP transfiere una representación del estado de dicho recurso en un formato comprensible para el receptor.

1.2 Arquitectura Cliente-Servidor y Esquema de Petición-Respuesta

HTTP es un protocolo de la capa de aplicación que se basa en una arquitectura de cliente-servidor operando sobre una conexión de capa de transporte confiable, habitualmente TCP. La comunicación se divide en dos roles principales: el cliente, denominado agente de usuario (user agent), que es el programa que establece la conexión con el fin de enviar peticiones HTTP —siendo el navegador web su ejemplo más conocido, aunque también incluye rastreadores web (web crawlers) y herramientas de línea de comandos—; y el servidor de origen (origin server), que acepta conexiones para dar servicio a las peticiones enviando respuestas.

HTTP es un protocolo orientado a transacciones, lo que significa que cada interacción sigue un ciclo cerrado: el cliente construye un mensaje de petición que comunica su intención mediante un método sobre un recurso identificado por una URI; el servidor recibe y analiza la petición interpretando la semántica del mensaje en relación con el recurso objetivo.

1.3 Naturaleza del Protocolo: HTTP como Protocolo sin Estado

HTTP se define fundamentalmente como un protocolo sin estado (stateless), lo que significa que cada mensaje de petición puede entenderse de forma aislada, sin depender de interacciones previas. El servidor no está obligado a retener ninguna información de sesión sobre el cliente entre una petición y la siguiente, considerando cada ciclo de petición-respuesta como una transacción independiente.

Dado que muchas aplicaciones web modernas requieren mantener un contexto —como carritos de compra o perfiles de usuario—, se utilizan mecanismos externos al protocolo base para simular un estado. Las cookies son el mecanismo principal: pequeños fragmentos de información que el servidor envía al cliente para que los almacene y los devuelva en peticiones posteriores.

1.4 Actores del Sistema: Agentes de Usuario y Servidores de Origen

Un agente de usuario es cualquier programa cliente que inicia una petición HTTP. Aunque el navegador web es el más conocido, existen muchos otros tipos: rastreadores web (web crawlers o spiders), herramientas de línea de comandos, aplicaciones móviles y dispositivos del Internet de las Cosas (IoT). Muchos agentes de usuario operan de forma autónoma en segundo plano realizando tareas como actualizaciones de firmware o indexación de contenido.

El servidor de origen es el programa capaz de generar respuestas con autoridad para un recurso determinado. Como dueño final del contenido o servicio, determina las reglas de acceso y la representación final del recurso solicitado.

2. Historia y Evolución de las Versiones

2.1 HTTP/0.9 (1991): La Versión Original

HTTP/0.9 fue la primera versión del protocolo, implementada por Tim Berners-Lee en 1991 como parte de la iniciativa original de la World Wide Web. Se caracterizó por una extrema simplicidad: soportaba únicamente el comando GET, carecía de cabeceras (headers), no incluía número de versión y las respuestas del servidor consistían exclusivamente en un flujo de bytes en formato HTML o texto plano.

2.2 HTTP/1.0 (1996): Introducción de Cabeceras y Metadatos

HTTP/1.0, documentado en el RFC 1945 en mayo de 1996, representó la primera gran revisión del protocolo. Sus innovaciones incluyeron la especificación de versión en el mensaje de petición, la introducción de cabeceras (headers) tanto en peticiones como en respuestas, y el soporte de tipos de contenido mediante tipos de medios similares al estándar MIME, lo que permitió transferir formatos de datos más allá del HTML.

2.3 HTTP/1.1 (1999): Conexiones Persistentes y el Estándar RFC 2616

HTTP/1.1 fue diseñado para corregir las deficiencias de rendimiento de las versiones anteriores, siendo su estandarización más célebre el RFC 2616 de 1999. Sus innovaciones clave incluyeron las conexiones persistentes (keep-alive), que mantienen la conexión abierta para múltiples peticiones eliminando la latencia del establecimiento de nueva conexión TCP por cada recurso; la segmentación de peticiones (pipelining), que permite enviar varias peticiones seguidas sin esperar la respuesta anterior; la transferencia codificada en bloques (chunked transfer encoding), para enviar datos de tamaño dinámico; y controles de caché más sofisticados con validadores como las etiquetas de entidad (ETags).

2.4 HTTP/2 (2015): Multiplexación y Optimización

HTTP/2, estandarizado en mayo de 2015 a través del RFC 7540, representó un cambio radical en la forma en que los mensajes se empaquetan y transportan, manteniendo intacta la semántica básica de HTTP. Su innovación más significativa fue la multiplexación: la capacidad de enviar y recibir múltiples mensajes de forma concurrente y asíncrona sobre una única conexión TCP, eliminando el problema del bloqueo en cabeza de línea (head-of-line blocking) a nivel de aplicación. Adicionalmente, introdujo la compresión de cabeceras mediante el algoritmo HPACK y la función de envío anticipado desde el servidor (server push), que permite al servidor enviar recursos proactivamente antes de que el cliente los solicite.

2.5 HTTP/3 (2018): El Cambio hacia QUIC y UDP

HTTP/3 es la evolución más reciente del protocolo, diseñada para superar las limitaciones de rendimiento impuestas por TCP. A diferencia de todas las versiones previas, utiliza QUIC (Quick UDP Internet Connections), un protocolo de transporte que opera sobre UDP. Esto permite la eliminación definitiva del bloqueo en cabeza de línea: al ser cada flujo de datos independiente, la pérdida de un paquete solo afecta a la petición específica a la que pertenece. Además, QUIC combina el establecimiento de la conexión de transporte y el protocolo de seguridad TLS en un solo paso, acelerando significativamente el inicio de la comunicación.

3. Funcionamiento Interno: Mensajes y Recursos

3.1 Representaciones de Recursos

En HTTP, los clientes no interactúan directamente con los recursos, sino con representaciones de su estado. Una representación es información destinada a reflejar un estado pasado, actual o deseado de un recurso dado, en un formato comunicable a través del protocolo. Se compone de metadatos —cabeceras que describen cómo interpretar los datos, como Content-Type o Content-Language— y datos de la representación, el flujo de bytes que se transfiere. Un único recurso puede tener múltiples representaciones válidas simultáneas: un documento disponible en varios idiomas o un conjunto de datos disponible en distintos formatos como JSON o XML.

3.2 Estructura y Sintaxis de los Mensajes

Un mensaje HTTP se divide en cuatro partes procesadas secuencialmente: (a) datos de control o línea inicial, que en una petición contiene el método, la URI del recurso y la versión del protocolo, y en una respuesta contiene la versión, el código de estado y una frase descriptiva; (b) sección de cabeceras, una tabla de metadatos en formato nombre/valor que termina con una línea en blanco; (c) contenido o cuerpo, el flujo de datos opcional que se envía después de las cabeceras.

3.3 Gestión de Sesiones mediante Cookies

Las cookies son pequeños fragmentos de datos que un servidor envía al cliente para que los almacene y los devuelva en peticiones posteriores, permitiendo así simular el estado en un protocolo sin estado. El servidor establece una cookie mediante la cabecera Set-Cookie en su respuesta; el agente de usuario la almacena y la devuelve automáticamente en cada petición posterior al mismo origen mediante la cabecera Cookie. Esto permite al servidor reconocer al usuario y recuperar su información de sesión.

3.4 Rol de los Intermediarios: Proxies, Puertas de Enlace y Túneles

HTTP permite el uso de intermediarios en la cadena de comunicación. El proxy es un agente de reenvío elegido por el cliente que recibe peticiones con URIs absolutas e intenta satisfacerlas, y se utiliza para seguridad, anonimato o caché compartida. La puerta de enlace (gateway) o proxy inverso actúa externamente como si fuera el servidor de origen, pero reenvía

las peticiones a servidores internos, empleándose para balanceo de carga o protección de servicios internos. El túnel actúa como un relé transparente entre dos conexiones sin modificar los mensajes, siendo fundamental para el método CONNECT.

4. Semántica de los Métodos de Petición

4.1 Clasificación: Seguridad e Idempotencia

Los métodos HTTP poseen propiedades semánticas que dictan el comportamiento esperado del servidor. Un método es seguro si su semántica es esencialmente de solo lectura: el cliente no solicita ni espera ningún cambio de estado en el servidor. Los métodos definidos como seguros son GET, HEAD, OPTIONS y TRACE. Un método es idempotente si el efecto intencional en el servidor de realizar múltiples peticiones idénticas es el mismo que si se realizara una sola vez. Todos los métodos seguros son también idempotentes.

4.2 Métodos de Recuperación: GET y HEAD

El método GET es el mecanismo principal para obtener información en HTTP. Solicita que el servidor transfiera una representación del estado actual del recurso identificado por la URI. Es un método seguro e idempotente, por lo que sus peticiones pueden ser procesadas por cachés, compartidas mediante URL y guardadas como marcadores. GET también permite solicitar rangos parciales de un recurso mediante la cabecera Range.

El método HEAD es idéntico a GET con la diferencia crítica de que el servidor no debe enviar el cuerpo en la respuesta. Se utiliza para obtener metadatos sobre un recurso sin consumir el ancho de banda de la transferencia completa.

4.3 Métodos de Envío y Procesamiento de Datos: POST y PUT

El método POST solicita que el servidor procese la representación enviada en la petición de acuerdo con la semántica propia del recurso objetivo. Es no seguro y no idempotente, por lo que repetir una petición POST puede tener efectos adicionales. Se utiliza típicamente para enviar formularios HTML, publicar contenido o completar transacciones. Si la operación resulta en la creación de un recurso nuevo, el servidor suele responder con el código 201 (Created).

El método PUT solicita que el estado del recurso de destino sea creado o reemplazado íntegramente por la representación enviada. A diferencia de POST, la URI identifica directamente el recurso que se quiere definir o actualizar. Es idempotente: ejecutarlo varias veces con el mismo contenido produce el mismo resultado.

4.4 Otros Métodos: DELETE, CONNECT, OPTIONS y TRACE

El método DELETE solicita que el servidor elimine la asociación entre el recurso de destino y su funcionalidad actual, de manera similar al comando rm de los sistemas Unix. Es idempotente pero no seguro. El método CONNECT solicita que un intermediario establezca un túnel TCP/IP hacia un servidor de destino, siendo fundamental para conexiones HTTPS a través

de proxies. El método OPTIONS permite al cliente solicitar información sobre las opciones de comunicación disponibles para un recurso. El método TRACE realiza un bucle de retorno del mensaje de petición, útil para diagnóstico.

5. Códigos de Estado y Respuestas del Servidor

5.1 Clasificación por Categorías

Los códigos de estado HTTP se clasifican en cinco categorías según el primer dígito: 1xx (respuestas informativas), 2xx (éxito), 3xx (redirecciones), 4xx (errores del cliente) y 5xx (errores del servidor).

5.2 Respuestas Informativas (1xx) y de Éxito (2xx)

Los códigos 1xx indican una respuesta provisional antes de que el servidor envíe la respuesta final. El código 100 (Continue) indica que el servidor ha recibido la parte inicial de la petición y que el cliente debe continuar enviando el resto. El código 101 (Switching Protocols) indica que el servidor acepta cambiar el protocolo de aplicación en la conexión actual.

Los códigos 2xx indican que la petición fue recibida, entendida y aceptada correctamente. El código 200 (OK) es el código de éxito estándar; el código 201 (Created) indica que la petición resultó en la creación de uno o más recursos nuevos.

5.3 Redirecciones (3xx)

La clase 3xx indica que el agente de usuario necesita realizar una acción adicional, generalmente una nueva petición a la dirección indicada en la cabecera Location. El código 301 indica un movimiento permanente; el 302, una redirección temporal; el 307 preserva el método HTTP al redirigir; el 308 es una redirección permanente que también preserva el método. El código 303 (See Other) redirige hacia un recurso diferente que actúa como respuesta indirecta. El código 304 (Not Modified) indica que el recurso no ha cambiado y el cliente debe usar su copia en caché.

5.4 Errores del Cliente (4xx) y del Servidor (5xx)

Los códigos 4xx indican que la petición parece contener un error atribuible al cliente. Entre los más relevantes: 400 (Bad Request) para peticiones con sintaxis malformada; 401 (Unauthorized) cuando se requiere autenticación; 403 (Forbidden) cuando el servidor se niega a cumplir la petición; 404 (Not Found) cuando no se encuentra el recurso; 405 (Method Not Allowed) cuando el método no es compatible; 410 (Gone) cuando el recurso ya no está disponible permanentemente.

Los códigos 5xx indican que el servidor ha fallado al procesar una petición que parece válida. El código 500 (Internal Server Error) es el código genérico de fallo del servidor.

6. Funcionalidades Avanzadas del Protocolo

6.1 Negociación de Contenido

La negociación de contenido es el mecanismo que permite servir diferentes representaciones de un mismo recurso según las preferencias del cliente. En la negociación proactiva (impulsada por el servidor), el agente de usuario expresa sus preferencias mediante cabeceras como Accept (formato), Accept-Encoding (compresión) y Accept-Language (idioma), y el servidor selecciona la representación más adecuada. En la negociación reactiva (impulsada por el agente), el cliente selecciona la representación tras recibir una respuesta inicial del servidor que lista las versiones disponibles, habitualmente con los códigos 300 (Multiple Choices) o 406 (Not Acceptable).

6.2 Peticiones Condicionales y Validadores

Una petición condicional incluye cabeceras que establecen una precondition que el servidor debe verificar antes de ejecutar el método solicitado. Los dos validadores principales son: la etiqueta de entidad (ETag), un identificador opaco asignado por el servidor a una versión específica del recurso que actúa como huella digital del contenido; y Last-Modified, una marca de tiempo que indica cuándo fue modificado el recurso por última vez. Las cabeceras de precondition más comunes son If-Match y If-None-Match, usadas con PUT o DELETE, y If-Modified-Since para la validación de caché.

6.3 Peticiones de Rango y Transferencia Parcial

Las peticiones de rango permiten solicitar únicamente una parte de la representación de un recurso, siendo útiles para la recuperación de transferencias interrumpidas y la optimización de ancho de banda. El servidor indica soporte mediante la cabecera `Accept-Ranges`; el cliente especifica el rango deseado mediante la cabecera `Range` (por ejemplo, `bytes=0-499` para los primeros 500 bytes); y el servidor indica qué porción se envía mediante `Content-Range`.

6.4 Almacenamiento en Caché

El sistema de caché de HTTP es un almacenamiento local de mensajes de respuesta previos que reduce el tiempo de respuesta y el consumo de ancho de banda. Una respuesta es fresca si está dentro de su período de validez, determinado por las cabeceras `Expires`, que proporciona una fecha absoluta de caducidad, y `Cache-Control`, que mediante la directiva `max-age` define la frescura en segundos relativos. Las directivas más importantes de `Cache-Control` son: `public` (la respuesta puede ser almacenada por cualquier caché), `private` (solo el navegador local puede almacenarla) y `no-cache` (se debe validar con el servidor antes de reutilizarla).

7. El Protocolo Seguro: HTTPS

7.1 Diferencias entre HTTP y HTTPS

HTTPS (HTTP Secure) no es un protocolo separado, sino el uso del protocolo HTTP ordinario sobre una conexión cifrada mediante TLS (Transport Layer Security). Las diferencias técnicas fundamentales son: HTTP transmite información en texto plano susceptible de interceptación, mientras que HTTPS utiliza cifrado bidireccional; HTTP opera por defecto en el puerto 80 y HTTPS en el puerto 443; HTTP carece de mecanismos propios para validar la identidad del servidor, mientras que HTTPS requiere certificados digitales X.509 firmados por una Autoridad de Certificación (CA) para tal fin.

7.2 Funcionamiento de TLS: Establecimiento de Conexión y Cifrado

La seguridad de HTTPS depende del protocolo TLS. Antes de que se transmita cualquier dato, cliente y servidor realizan un proceso de establecimiento de conexión (handshake en inglés, término sin traducción equivalente precisa en el estándar) que cumple tres funciones: negociación de la versión de TLS y los algoritmos de cifrado; autenticación mediante el certificado digital X.509 del servidor verificado contra las autoridades de certificación de confianza del cliente; e intercambio de claves para generar una clave de sesión compartida. El cifrado asimétrico (de clave pública) se utiliza únicamente durante este proceso inicial, mientras que el cifrado simétrico, computacionalmente más eficiente, se emplea para el resto de la sesión.

7.3 Infraestructura de Clave Pública (PKI) y Certificados X.509

La Infraestructura de Clave Pública (PKI, por sus siglas en inglés Public Key Infrastructure) es el conjunto de roles, políticas y procedimientos necesarios para crear, gestionar

y revocar certificados digitales. Un certificado X.509 es un documento electrónico que vincula una clave pública con una identidad, conteniendo la clave pública del servidor, información sobre el propietario, la firma digital de la autoridad emisora y el período de validez. Las Autoridades de Certificación (CA) actúan como notarios digitales: los navegadores y sistemas operativos incluyen de fábrica una lista de certificados raíz de CA de confianza, formando una cadena de confianza que permite validar cualquier certificado firmado por ellas.

8. Consideraciones de Seguridad y Privacidad

8.1 Ataques de Intermediario (Man-in-the-Middle)

Las conexiones HTTP no cifradas son susceptibles a ataques de intermediario (man-in-the-middle o MITM), en los que un atacante interpone entre cliente y servidor para interceptar o modificar el tráfico. HTTPS está diseñado específicamente para resistir este tipo de ataques garantizando la autenticidad del servidor mediante certificados digitales.

8.2 Vulnerabilidades Comunes

Entre las vulnerabilidades más relevantes se encuentran la inyección de comandos y la inyección SQL (SQL injection), que explotan una validación insuficiente de los datos de entrada; y los ataques de longitud, que manipulan las cabeceras de tamaño del contenido para confundir al servidor. La capa TLS puede verse comprometida por el ataque denominado vaciado SSL (SSL stripping), que fuerza una degradación de HTTPS a HTTP.

8.3 Privacidad y Huella Digital del Navegador

Las cabeceras de negociación de contenido (como Accept-Language o User-Agent) pueden ser utilizadas para identificar y rastrear usuarios mediante la técnica denominada huella digital del navegador (browser fingerprinting), que combina múltiples características del agente de usuario para crear un identificador único sin necesidad de cookies.

9. Documentación Técnica y Estandarización

9.1 El Rol de los Documentos RFC

Los documentos RFC (Request for Comments, Solicitudes de Comentarios) son publicaciones formales del Grupo de Trabajo de Ingeniería de Internet (IETF, Internet Engineering Task Force) que definen los estándares de interoperabilidad de Internet. Cada versión principal de HTTP tiene su propio RFC de referencia: RFC 1945 para HTTP/1.0, RFC 2616 para HTTP/1.1, RFC 7540 para HTTP/2 y RFC 9114 para HTTP/3.

9.2 RFC 9110: La Especificación Actual de la Semántica HTTP

El RFC 9110 constituye la especificación vigente sobre la semántica base de HTTP, independientemente de la versión del protocolo. Define los métodos, códigos de estado, cabeceras y comportamientos que son comunes a todas las versiones de HTTP, estableciendo el

fundamento semántico sobre el que se construyen las especificaciones de transporte de cada versión.

9.3 Limitaciones de HTTP/1.1 y el Origen de HTTP/2

A pesar de su dominio durante más de 15 años, HTTP/1.1 presentaba limitaciones de rendimiento evidentes ante la creciente complejidad de las páginas web: el bloqueo en cabeza de línea impedía que peticiones posteriores avanzaran mientras una petición lenta se procesaba; los navegadores debían abrir múltiples conexiones TCP paralelas para mitigar este problema, consumiendo recursos adicionales; las cabeceras se transmitían en texto plano en cada petición, repitiendo metadatos sin cambios; y cada nueva conexión requería el proceso de establecimiento TCP y TLS, añadiendo latencia. HTTP/2 surgió para resolver estos problemas manteniendo la misma semántica central.